

Query Parameters and Validation

Section Review

Query Parameters I

- A **query parameter** is a key-value pair attached to the end of a URL by the client.
- FastAPI can extract the values and pass them in as function arguments to the route handlers.
- We can use type annotation to attempt type coercion. FastAPI will raise an error if it fails.

Query Parameters II

- Query parameters can be optional or required.
- When optional, a common practice is to assign a default argument of **None**. This change also requires updating the type annotation to include **| None**.
- Optionality is generally preferred; it makes the endpoint more flexible.

The Annotated Type

- Python's **Annotated** type declares a type and an associated piece of metadata.
- FastAPI uses the associated metadata to perform additional validations at runtime.
- The *first value* within the square brackets will be the inferred type for the parameter.
 - value: **Annotated[T, SomeValidator()]**

The Query Function

- Use **Query()** as the second value to **Annotated** to tell FastAPI that the data can be parsed from a query parameter.
- The advantage of a plain **str** or **int** is that **Query** can perform additional validations and transformations.
- **Query** can check for min length, max length, numeric boundaries, and more.

The AfterValidator Function

- The **Annotated** type can accept multiple validators after the first type.
- Pass them all within the square brackets. They will be run in order.
- The **AfterValidator** function from Pydantic runs a custom validator function after the initial **Query** validation.

Reusability

- Validators like **Query()** can be extracted to helper variables to enable reusability across multiple routes.
- A custom **Annotated** type can also be assigned to a reusable variable.
- Even with these refactors, we have to declare *one route handler function parameter* per query parameter.

Pydantic Models I

- A Pydantic model can perform validations on multiple fields (pieces of related data).
- For example, a Pydantic model can capture all of our query parameters or the shape of a payload from the client.
- The **Query** function is using Pydantic under the hood. It's associating the validation with a query parameter.

Pydantic Models II

- Create a class that inherits from Pydantic's **BaseModel** superclass.
- Declare class attributes to represent one piece of data. Use regular type annotation (**str | None**).
- To add validations beyond type coercion, assign the class attribute to a **Field** value.
- **Field** accepts the same parameters as **Query**.

Swagger API Documentation

- Use **tags** to group related routes under a shared header.
- Use **summary** and **description** to provide information about a route.
- Use **deprecated** to mark an endpoint as expired.
- Use **openapi_tags** to provide descriptions for tags.